

Supplier Master Data Automation and Approval

Prepared for Chinmayee Surwade

Email:Chinmayee.surwade@gmail.com

Phone:9363099166

This project addresses a common operational problem: supplier requests arrive with missing information, inconsistent formatting, and possible duplicates. The exercise validates each request, separates review cases, applies simulated regional decisions, and writes only approved suppliers to the output master.

Completed result

The local Python implementation processed 40 synthetic requests against 10 existing suppliers. It produced 17 approved new suppliers, nine validation failures, eight duplicate-review cases, three rejections, and three pending approvals. The final master contains 27 suppliers. Five automated tests passed. A separate forecast exercise compared two baselines on four held-out weeks.

Execution boundary

The runnable implementation uses Python and CSV files. Approval decisions are synthetic inputs, not decisions from people. UiPath, Automation Anywhere, Power Automate, and Qlik Sense were not executed. Their sections are build guides. Global handoffs are a proposed operating model, not evidence of coordinating actual regional teams. The project demonstrates implemented logic and supporting BA work; native hands-on claims require platform execution.

Included files

The companion GitHub package contains the source, sample data, output master, request results, audit log, tests, forecast results, README, platform guides, resume wording, interview explanation, and this Word case study. No confidential supplier data or credentials are included.

Business problem scope and requirements

The fictional business maintains a supplier master used by procurement. Requesters send new supplier details through a standard intake file. Staff must validate the fields, avoid duplicate identities, obtain approval, and preserve evidence of the decision. The initial scope covers new supplier creation only.

ID	Requirement	Acceptance condition
FR01	Standardize incoming values.	Trim names, collapse repeated spaces, uppercase country and registration ID, lowercase email.
FR02	Validate required fields and format.	Reject missing name, country, registration ID, email, or request ID; check email syntax and country list.
FR03	Detect existing and batch identities.	Country plus registration ID matches enter review; no automatic merge.
FR04	Route valid requests by region.	IN → APAC; DE → EMEA; US → AMERICAS.
FR05	Apply approval before master creation.	Only approved, valid new identities enter the output master.
FR06	Preserve decision evidence.	Log request ID, event, outcome, reason, actor role, and record fingerprint.
FR07	Make repeat execution predictable.	The same inputs produce the same master and summary without extra rows.
FR08	Reconcile every request.	Final status counts total 40; master growth equals approvals.

Out of scope

Bank-account validation, tax-law verification, production ERP posting, sanction screening, fuzzy automatic merging, supplier amendments, deletion, and real notifications. Country and registration ID are a simplified matching key; a real MDM design needs identifier type, legal-entity rules, jurisdiction, and source authority.

Business and technical errors

Missing data and duplicate candidates require a business response. A failed file read or unavailable application is a technical failure. Do not retry invalid data indefinitely. A production bot should retry transient technical faults within limits and retain a clear failure record.

Process and data model

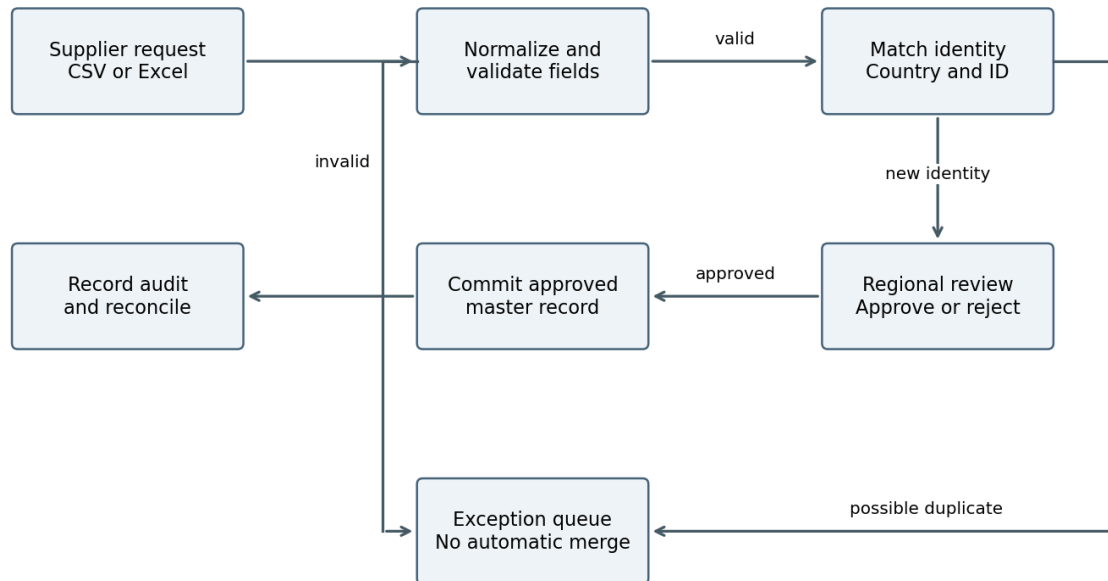


Figure 1 Implemented logical flow. Regional decisions are simulated; exception ownership and human review are proposed.

The executed model has one row per existing supplier, one row per incoming request, one row per simulated decision, and one row per audit event. A request may generate a validation event and a decision event. The request result has one final outcome per request. The final master contains the original supplier rows plus approved new identities.

Object	Key and important fields
Supplier master	supplier_id; supplier_name; country; registration_id; contact_email.
Request	request_id; supplier_name; country; registration_id; contact_email.
Decision	request_id; region; decision; approver_role; reason.
Result	request_id; status; reason; region.
Audit event	request_id; event; status; reason; actor; fingerprint.

The fingerprint is a SHA-256 digest of the normalized request. It helps identify the exact record used in this deterministic exercise. It does not make the CSV audit tamper-proof. Production audit needs controlled timestamps, durable storage, access controls, retention, and integrity protections.

Master data definitions and matching rules

Term	Meaning and rule
Supplier	Approved business identity represented in the master.
Master record	Current accepted representation of a supplier for the scoped attributes.
Identity key	Country plus registration ID in this simplified exercise.
Golden record	Consolidated trusted representation based on approved source and survivorship decisions; full golden-record consolidation is not implemented here.
Duplicate candidate	A request sharing an identity key with a master row or an earlier valid request. Review required.
Survivorship	Policy deciding which source value to retain when records conflict. Proposed: verified legal source outranks unverified intake text.
Data steward	Reviews meaning, quality issues, and proposed corrections.
Data owner	Accountable for supplier data definitions and permitted use.
Approval	Recorded authorization to add a valid new identity.
Reconciliation	Agreement between request counts, result counts, and approved master additions.

Normalization and validation

Whitespace is trimmed and repeated spaces in names are collapsed. Country and registration ID are uppercased; email is lowercased for the exercise. Supported countries are IN, DE, and US. Required fields must be present. Email checks syntax only and do not confirm deliverability. Registration IDs are fictional and are not checked against a tax registry.

Duplicate behavior

Existing identity matches and repeated batch identities are flagged. If the earlier candidate is rejected or pending, a later matching request still remains in duplicate review because it needs context. Similar names with different IDs are not matched by this implementation. A real steward would review name similarity alongside reliable identifiers.

Updates and survivorship

No automatic overwrite or merge occurs. The exercise only appends approved new suppliers. A future change request must identify the current master version, proposed values, evidence, approval, and effective date. Bank details should use a separate restricted process with stronger verification.

Executed dataset and results

Outcome	Count	Share of 40 requests
APPROVED	17	42.5%
VALIDATION_FAILED	9	22.5%
DUPLICATE_REVIEW	8	20.0%
REJECTED	3	7.5%
PENDING_APPROVAL	3	7.5%
Total	40	100.0%

How the sample was constructed

R001–R004 have missing names. R005–R007 have invalid emails. R008–R009 use unsupported countries. R010–R014 match existing identities. R016, R018, and R020 repeat earlier batch identities. Of the 23 valid new candidates, the decision file approves 17, rejects three, and supplies no decision for three. The faults are intentional; these rates do not estimate real supplier-data quality.

Worked requests

R015 is a valid US request. Its name spacing, email case, country, and registration ID are normalized. A simulated AMERICAS decision approves it, creating a new master row. R016 shares R015's identity and enters duplicate review. R038 has valid input but no decision, so it remains pending and creates no master row.

Reconciliation

40 results equal 40 input requests. $17 + 9 + 8 + 3 + 3 = 40$. Original master 10 + approvals 17 = final master 27. Repeating the batch regenerates the same outputs from the same inputs. This is deterministic file regeneration, not a transactional incremental ERP integration.

Evidence files

outputs/request_results.csv lists every outcome. outputs/approved_master.csv contains 27 rows. outputs/audit_log.csv records validation and simulated decisions. outputs/summary.json records aggregate results and the execution boundary.

UiPath build guide

This section maps the executed rules to a native UiPath workflow. No UiPath project export or run is included. Complete and run the workflow in your own authorized environment before listing hands-on UiPath project experience.

Stage	Suggested implementation	Evidence to capture
Intake	Read the CSV inputs into tables. Validate headers before processing.	Input schema check and file path configuration.
Normalize	For each request, trim strings, collapse name spaces, and standardize case.	Before-and-after sample values.
Validate	Use conditions for required fields, allowed country, and email syntax.	Invalid row output with reasons.
Match	Compare identity keys with the master and a batch identity set.	Existing and batch duplicate tests.
Review	Write eligible requests to a review queue or approved review interface.	Pending state and reviewer evidence.
Commit	After approved decision, write the supplier to a controlled target.	Approved-only additions and reconciliation.
Handle faults	Use scoped exception handling and bounded retry for transient errors.	Failure log and restart test.

Implementation sequence

First reproduce the CSV validation exercise without ERP UI automation. Confirm all 40 statuses match the Python reference. Then add a small mock supplier form and test stable element targeting, page waits, and error detection. Store configuration separately. Keep credentials outside the project. Do not add real ERP posting until its approval and duplicate controls are tested.

Platform acceptance tests

Missing field; invalid format; existing duplicate; repeated batch identity; approval; rejection; pending decision; file unavailable; target write failure; and restart after interruption. Capture actual run logs and screenshots. The reference implementation cannot substitute for these platform results.

Automation Anywhere and Power Automate guides

Automation Anywhere equivalent

Build the same CSV intake and normalization logic, keeping the identity key and outcome vocabulary unchanged. Use the platform's available table, string, conditional, loop, file, and error-handling capabilities. Match results with the reference CSV by request ID, not row order. Export the native bot and capture its run evidence after testing. No native bot is supplied here.

Power Automate approval design

A proposed cloud flow starts when an eligible request is added to an approved intake list. Check whether it has already been evaluated, map its country to the approved regional routing configuration, and submit an approval. Microsoft documents the Start and wait for an approval action. Once a decision arrives, store the outcome and reason, then call the controlled master-update service only for approved requests.

Condition	Proposed action
Invalid or duplicate request	Do not submit approval; route to steward review.
Valid new request	Record pending state and request approval from the configured regional role.
Approved	Recheck identity and request version before committing.
Rejected	Store reason; keep master unchanged.
No decision or timeout	Remain pending or escalate under an agreed timer. No auto-approval.
Flow retry or duplicate trigger	Use request ID and source version as idempotency key.
Changed request during review	Invalidate stale decision and require review of the new version.

Keep execution claims accurate

The package contains a local approval simulation and build specifications. It contains no cloud-flow export, connector configuration, UiPath workflow, or Automation Anywhere bot. If the target role requires those tools hands-on, run the designs in the actual platforms and retain evidence before adding them as implemented skills.

Regional handoff and governance

The regional model routes IN to APAC, DE to EMEA, and US to AMERICAS. These labels are used by the local code to validate simulated decision routing. No people in those regions participated.

Work	Proposed owner	Required handoff
Input correction	Requester and regional steward	Request ID, failed fields, required evidence.
Duplicate resolution	Master Data Steward	Matching identifiers, current master reference, decision reason.
New supplier approval	Regional business approver	Validated request version, risk context, and evidence.
Master commit	Technical service owner	Approved request ID, version, identity key, and acknowledgement.
Operational escalation	Global process owner	Open issues, age, impact, responsible queue, next action.

Practical cross-region plan

Store event timestamps in UTC and show local time only for convenience. Agree each team's working calendar and overlap window. At shift handoff, record the request IDs awaiting decisions, attempted actions, blockers, and next owners. Keep a shared decision log. A regional label alone is not evidence of global team coordination.

Workshop agenda and decisions

A proposed 45-minute workshop covers required fields, identity keys, duplicate handling, approval roles, escalation, and audit expectations. Use R015 and R016 to show why a matching identity needs review. Ask whether the identifier type and jurisdiction should be added to the identity key. Record owner approval before production use. This workshop pack has been prepared but no workshop was held.

Change and training

Pilot with requesters and stewards, demonstrate three cases, and have participants submit a valid request, correct an invalid request, and explain a duplicate flag. Record task completion and feedback. Measure adoption through correct task execution and fewer repeated errors. Attendance and notifications alone do not prove adoption.

Testing and limitations

Test	What was checked	Result
Normalization	Name spaces, country and ID case, email case.	Passed
Validation	Unsupported country and invalid email create explicit failures.	Passed
Identity scope	Existing key is flagged; same ID in a different country remains distinct.	Passed
Batch duplicates	Second valid request with the same identity enters review.	Passed
Full run and repeat	Expected 40 outcomes, 27 master rows, unique identities, pending cases, and identical repeat summary.	Passed

Controls implemented

The code rejects missing or repeated request IDs, duplicate decision records, unknown decision codes, wrong approval regions, and missing decision reasons. These guards are present in the code, but the five-test suite does not separately exercise every guard. Extend the tests before production use.

Known technical limitations

The implementation assumes controlled CSV headers and regenerates outputs instead of maintaining a persistent transaction store. It does not handle concurrency, partial writes, credentials, real approver identity, approval expiration, request amendments, file locks, or adversarial inputs. The audit includes fingerprints but no real event timestamps. Real bots and cloud flows need security, durable queues, monitoring, and recovery tests.

Responsible interpretation

The test results show that the reference rules work on designed cases. They do not establish enterprise readiness, platform proficiency, or business adoption. The value for a recruiter is the clear link from requirements to implemented rules, output evidence, and known boundaries.

Reporting and forecasting exercise

Use Power BI or another available BI tool to load request_results.csv, approved_master.csv, and audit_log.csv. Keep the request result at one row per request. The audit is one row per event, so joining it directly can inflate request counts. The package provides source outputs and a reporting specification, not a live BI file.

View	Definition and decision
Request status	Counts by final status, with distinct request_id denominator.
Validation reasons	Requests failing each reason. Multi-reason totals may exceed affected requests.
Regional review	Valid requests by region and outcome. Do not equate regional volume with team performance.
Master growth	Approved new rows and original master count, reconciled with results.
Duplicate review	Existing identity matches versus batch identity matches.
Audit coverage	Requests with validation event; approved requests with simulated decision event.

Forecast method actually run

A separate synthetic series contains 16 weekly request totals. Weeks 1–12 are training data; weeks 13–16 are held out. The actual holdout values are 43, 48, 45, and 50. Both methods forecast from the same origin and do not read holdout data during training. This series is independent of the 40-request batch.

Method	Forecast for each holdout week	MAE
Last observed week	46, 46, 46, 46	2.5 requests
Mean of last four training weeks	42.5, 42.5, 42.5, 42.5	4.0 requests

Mean absolute error is the mean of the four absolute differences. The last-week baseline performs better in this small fixture. Four holdout points are insufficient to establish a useful business forecast. Real planning needs longer history, calendar effects, backtesting across origins, uncertainty, and capacity assumptions.

Qlik Sense option

The reporting tables and definitions can be reused in Qlik Sense. A Qlik dashboard is optional until the JD requires it specifically. Do not claim Qlik development from a generic dashboard specification.

User guide and recruiter explanation

Run the reference implementation

```
python generate_data.py
python supplier_pipeline.py
python test_pipeline.py
python forecast.py
```

Run these commands from the extracted project folder using Python 3. The implementation uses only the standard library. `generate_data.py` creates or overwrites the synthetic inputs, while `supplier_pipeline.py` overwrites the output files. For your own copies, preserve input files before rerunning the generator.

Review and change a simulated decision

Open `request_results.csv` and find pending request R038. In a copy of `approval_decisions.csv`, add a decision with R038, its correct region, APPROVED or REJECTED, a simulated role, and a reason. Run the pipeline again and inspect the result and master count. The bundled tests expect the original decisions and should be run against the original fixture.

Interview explanation

“I built a supplier master-data automation exercise with validation, duplicate review, and approval controls. The Python reference processed 40 requests and added 17 approved suppliers to the master. I tested normalization, identity matching, and repeat runs. I also mapped the process to UiPath, Automation Anywhere, and Power Automate, but those native designs still need platform execution.”

Resume project entry

Supplier Master Data Automation and Approval | Independent portfolio project

Built and tested a Python workflow to validate 40 supplier requests, flag duplicate identities, apply simulated regional approvals, and produce an approved master with audit outputs. Documented RPA build guides, governance controls, and reporting definitions; compared two forecasts on held-out synthetic data.

How this addresses the gaps

It adds actual automation and MDM logic to your portfolio. It supports forecasting practice and shows a regional handoff design. It does not replace real-job MDM, actual global coordination, or native UiPath, Automation Anywhere, Power Automate, and Qlik execution. Those claims must come from separately completed work.

References and GitHub package

Public documentation used

Microsoft Learn. Create and test an approval workflow with Power Automate.

<https://learn.microsoft.com/en-us/power-automate/modern-approvals>

Microsoft Learn. Get started with Power Automate approvals.

<https://learn.microsoft.com/en-us/power-automate/get-started-approvals>

UiPath Academy. Automation Explorer.

<https://academy.uipath.com/learning-plans/automation-explorer>

Automation Anywhere. Community Edition.

<https://www.automationanywhere.com/products/enterprise/community-edition>

Documentation checked 5 October 2026. Account eligibility, available activities, deployment model, and licensing must be checked in the chosen native environment. The platform steps here are implementation guides rather than validated native exports.

Package structure

File or folder	Purpose
supplier_pipeline.py	Executable normalization, validation, identity matching, simulated approval, and output logic.
generate_data.py and data	Deterministic synthetic fixture and three core input CSVs.
test_pipeline.py	Five tests for the implemented reference.
forecast.py	Fixed-origin holdout baseline comparison.
outputs	Results, approved master, audit, summary, and forecast results.
README.md	Run instructions, findings, and accurate execution status.
PLATFORM_GUIDES.md	Native build sequence and acceptance checks.
RESUME_AND_INTERVIEW.md	Project wording and concise explanation.
Word case study	Business context, implementation, evidence, and limitations.